

TP1 — Rendimiento de Computadoras

Docente: Javier Alejandro Jorge

Alumnos: Acuña María Pía

Di Paolo Valentino

Grupo: DontReadMe

1. Benchmarks y tareas diarias

Tabla de tareas diarias y benchmarks

Tarea diaria	Benchmark
Navegación web	Speedometer
Compilar código	Timed GCC Compilation
Gaming	Unigine Heaven
Simulación Vivado	pts/build-linux-kernel
Comprimir/extraer archivos (backups, .tar.gz, descargas)	7-Zip Compression

Especificaciones de las computadoras

Computadora — Valentino Di Paolo
Intel Core i7-1165G7 11th gen, 2.8 GHz, 4 núcleos
16 GB RAM DDR4
Disco SSD
Intel Iris Xe Graphics

Computadora — Acuña María Pía
AMD Ryzen 7 7445HS w/ Radeon 740M Graphics (3.20 GHz)
16,0 GB RAM DDR5
Disco SSD
GeForce RTX 40

1.4 Comparación de resultados (7-Zip Compression)

	Valentino	Pía
CPU	i7-1165G7 @ 4.70 GHz, 4C/8T	Ryzen 7 7445HS @ 3.19 GHz, 6C/12T
RAM	16 GB	16 GB

	Valentino	Pía
Compresión	28.930 MIPS	29.649 MIPS
Descompresión	25.808 MIPS	41.187 MIPS

Los resultados muestran una diferencia más marcada en descompresión (25.808 MIPS en la i7-1165G7 de Valentino contra 41.187 MIPS en la Ryzen 7 7445HS de Pía) que en compresión (28.930 vs 29.649 MIPS, prácticamente parejo). Esto es consistente con la diferencia de núcleos e hilos entre ambos procesadores (4C/8T vs 6C/12T): la descompresión con 7-Zip escala mejor con más hilos disponibles, mientras que la compresión está más limitada por el rendimiento por núcleo y por la frecuencia individual, donde la diferencia entre ambas CPUs es menor, entonces para la tarea de comprimir/extraer archivos, tener más núcleos ayuda notablemente cuando la carga es paralelizable (descompresión), pero no alcanza a marcar una diferencia tan grande cuando el algoritmo depende más de un solo hilo (compresión).

```
tino@tino-Smart-Pro-Q7-i7:~/tp1-rendimiento$ 7z b
7-Zip 26.00 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-02-12
64-bit locale=es_ES.UTF-8 Threads:8 OPEN_MAX:1024, ASM

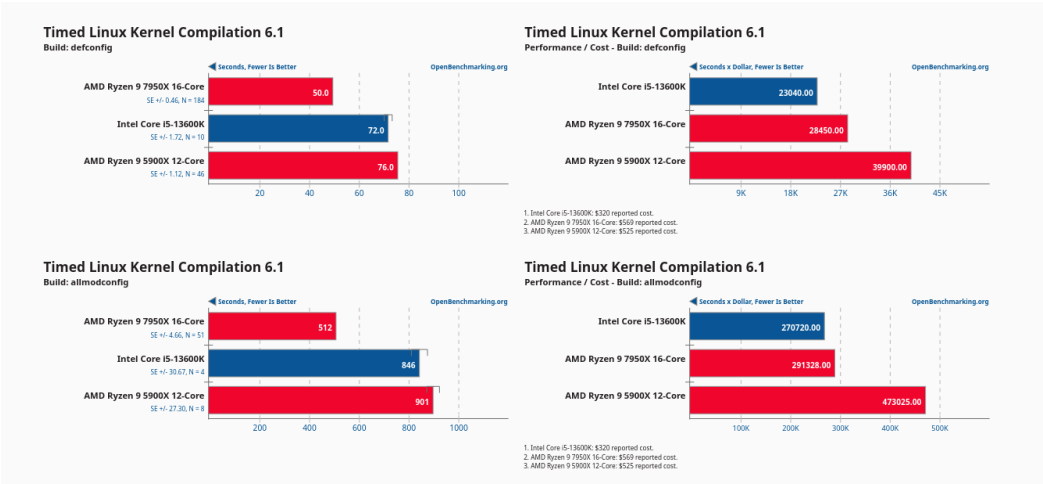
Compiler: ver:15.2.0 GCC 15.2.0 : SSE2
Linux : 7.0.0-30-generic : #30-Ubuntu SMP PREEMPT_DYNAMIC Fri Jul 31 18:22:54 UTC 2026 : x86_64
PageSize:4KB THP:madvise hwcaps:6 hwcaps2:2
11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz
(806C1)

1T CPU Freq (MHz): 4518 4393 4411 4254 4449 4459 4356
4T CPU Freq (MHz): 394% 3951 396% 3722
8T CPU Freq (MHz): 661% 2716 726% 3004

RAM size: 15253 MB, # CPU hardware threads: 8
RAM usage: 1779 MB, # Benchmark threads: 8

Dict  Speed Usage  Compressing  |  Decompressing
      KiB/s  %      R/U Rating  |  Speed Usage  R/U Rating
                        MIPS  MIPS  |  KiB/s  %      MIPS  MIPS
-----|-----
22:   29276  651  4375  28480 |  322112  697  3938  27467
23:   27534  658  4265  28054 |  328402  736  3860  28407
24:   27453  687  4298  29518 |  312253  716  3829  27397
25:   25985  677  4386  29669 |  224316  666  2995  19959
-----|-----
Avr:   27562  668  4331  28930 |  296771  704  3655  25808
Tot:           686  3993  27369
```

2. Rendimiento en la compilación del kernel de Linux



Tiempos de compilación

- AMD Ryzen 9 7950X: 50,0 s
- Intel Core i5-13600K: 72,0 s
- AMD Ryzen 9 5900X: 76,0 s

Precios de referencia

- Intel Core i5-13600K: \$320
- AMD Ryzen 9 5900X: \$525
- AMD Ryzen 9 7950X: \$569

Tabla comparativa

	i5-13600K	Ryzen 9 5900X	Ryzen 9 7950X
Tiempo (kernel 6.1 defconfig)	72 s	76 s	50 s
Núcleos	14	12	16
Rendimiento (1/t)	0,01389	0,01316	0,02000
Precio	\$320	\$525	\$569
TDP (W)	125	105	170
Eficiencia/núcleo	0,000992	0,001096	0,001250
Eficiencia/dinero	0,0000434	0,0000251	0,0000352
Eficiencia/energía	0,0001111	0,0001253	0,0001176

Conclusiones

- Rendimiento i5 vs 5900X: $i5 = 0,01389$ vs $5900X = 0,01316 \rightarrow$ el i5 rinde ~5,5% más.
- Speedup del 7950X: $1,44\times$ respecto al i5-13600K; $1,52\times$ respecto al Ryzen 9 5900X.
- Mejor uso de núcleos: AMD Ryzen 9 7950X, con 0,00125 de rendimiento por núcleo.
- Más eficiente en costo — dinero: Intel i5-13600K (menor relación segundos $\times$ dólar, 23.040, según el panel de openbenchmarking).
- Más eficiente en costo — energía: AMD Ryzen 9 5900X (menor TDP, 105 W).

3. ESP32

print de esp32

El código utilizado para la realización de este punto está cargado en el repositorio de GitHub

Calibrando N para que cada bucle tarde ~10 s a 240 MHz...

```
N_int = 217366259
```

```
N_float = 159402051
```

```
N_double = 15494881
```

```
=== RESULTADOS (formato CSV) ===
```

```

freq_MHz, tipo, N, tiempo_ms

240, int, 217366259, 9999.98

240, float, 159402051, 9999.97

240, double, 15494881, 10002.18

160, int, 217366259, 15028.09

160, float, 159402051, 15028.01

160, double, 15494881, 15032.30

80, int, 217366259, 30226.14

80, float, 159402051, 30227.00

80, double, 15494881, 30240.08

40, int, 217366259, 61143.80

40, float, 159402051, 61143.87

40, double, 15494881, 61194.61

```

Frecuencia	int	float	double
240 MHz	9.999,98	9.999,97	10.002,18
160 MHz	15.028,09	15.028,01	15.032,3
80 MHz	30.226,14	30.227,01	30.240,08
40 MHz	61.143,8	61.143,87	61.194,61

se realizaron graficos con python y ayuda de la ia

gráfico tiempo vs frecuencia

Tiempo de ejecución vs Frecuencia de CPU (ESP32) N calibrado para ~10s a 240 MHz

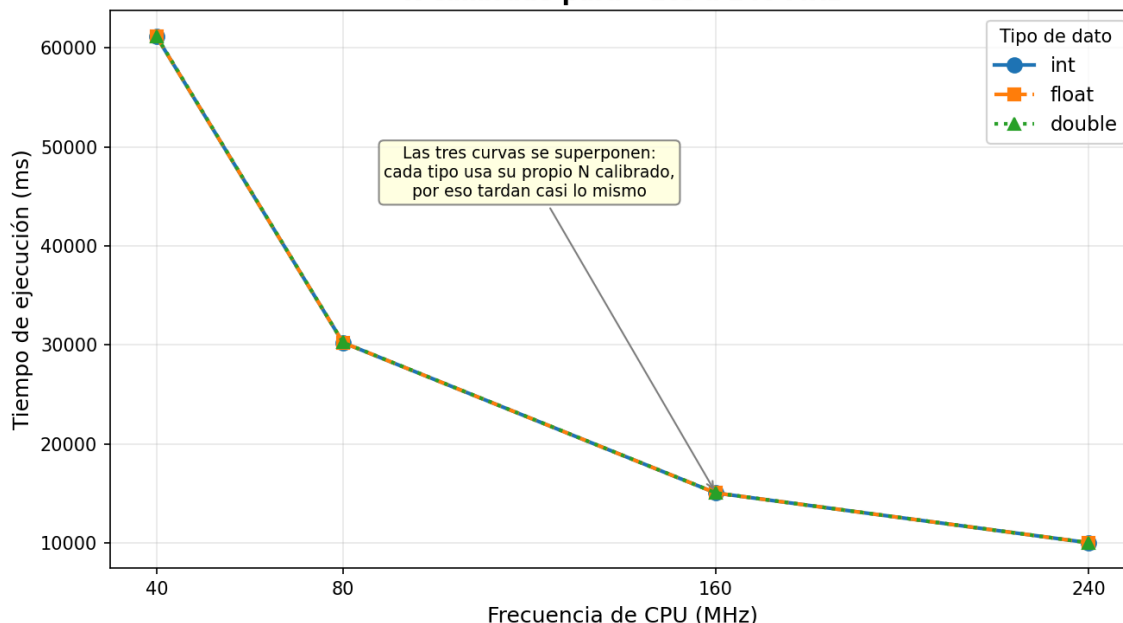
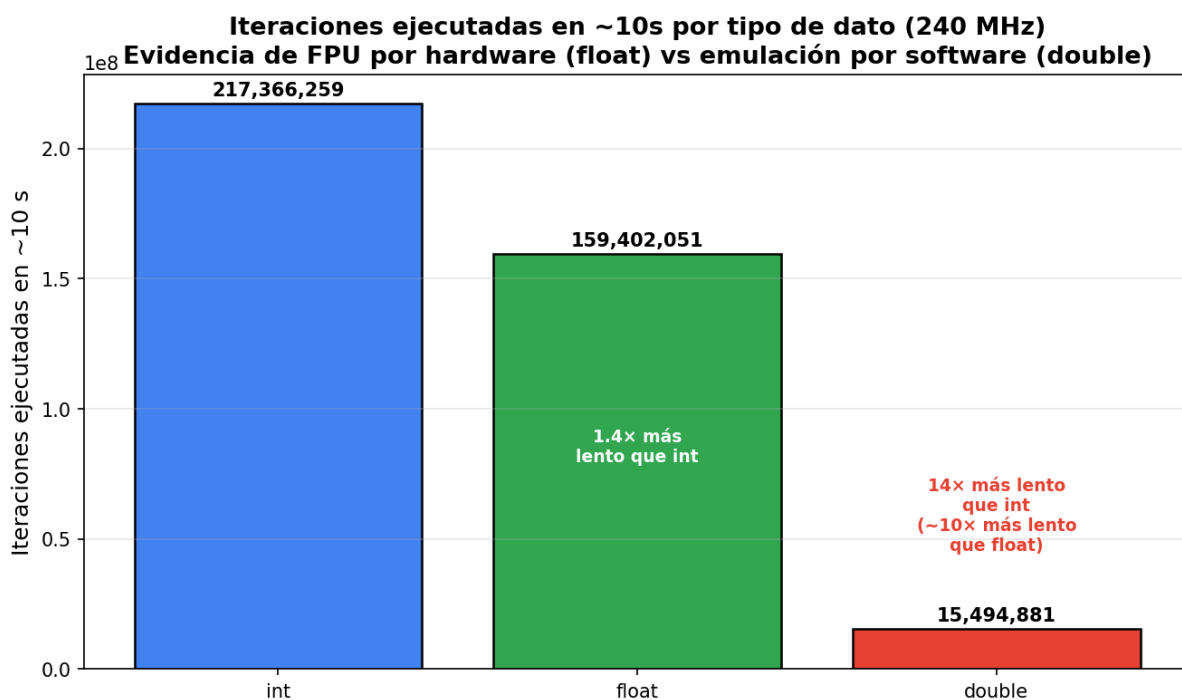


gráfico tiempo vs frecuencia



En este punto se analizó el efecto de la frecuencia de reloj del ESP32 sobre el tiempo de ejecución, ejecutando tres bucles equivalentes (con tipos de dato int, float y double) a 240, 160, 80 y 40 MHz. El número de iteraciones de cada bucle se calibró para que tardara aproximadamente 10 segundos a la frecuencia máxima (240 MHz).

Efecto de la frecuencia: el tiempo de ejecución resultó inversamente proporcional a la frecuencia de CPU. Al reducir la frecuencia a la mitad, el tiempo se duplicó (por ejemplo, de 80 a 40 MHz el tiempo pasó de ~30.200 ms a ~61.150 ms). Esto se verifica en que el producto tiempo $\times$ frecuencia se mantiene aproximadamente constante en todas las mediciones, confirmando que los bucles son CPU-bound y validando la relación teórica $T = N \times CPI \times (1/f)$. Se observó un leve incremento del producto al bajar la frecuencia (~2%), atribuible a que los accesos a memoria no escalan exactamente igual que la frecuencia del núcleo.

Diferencia entre tipos de dato: para tardar el mismo tiempo a 240 MHz, el bucle de enteros ejecutó 217 millones de iteraciones, el de float 159 millones y el de double solo 15 millones. Es decir, double resultó aproximadamente 14 veces más lento que int y unas 10 veces más lento que float. Esto se debe a que el ESP32 (Xtensa LX6) posee una unidad de punto flotante (FPU) por hardware únicamente para precisión simple (float), mientras que las operaciones de doble precisión (double) se emulan por software, lo que las hace mucho más costosas. La aceleración por hardware explica por

qué float, pese a ser de punto flotante, se acerca al rendimiento de los enteros, mientras que double queda muy por detrás.

4. Time profiling — gprof

Di Paolo Valentino

index	% time	self	children	called	name
<spontaneous>					
[1]	100.0	0.03	22.99		main [1]
		7.80	7.62	1/1	func1 [2]
		7.57	0.00	1/1	func2 [4]

		7.80	7.62	1/1	main [1]
[2]	67.0	7.80	7.62	1	func1 [2]
		7.62	0.00	1/1	new_func1 [3]

		7.62	0.00	1/1	func1 [2]
[3]	33.1	7.62	0.00	1	new_func1 [3]

		7.57	0.00	1/1	main [1]
[4]	32.9	7.57	0.00	1	func2 [4]

```
gprof -a test_gprof gmon.out > analysis.txt
```

index	% time	self	children	called	name
<spontaneous>					
[1]	100.0	0.03	22.99		main [1]
		15.37	7.62	2/2	func1 [2]

		15.37	7.62	2/2	main [1]
[2]	99.9	15.37	7.62	2	func1 [2]
		7.62	0.00	1/1	new_func1 [3]

		7.62	0.00	1/1	func1 [2]
[3]	33.1	7.62	0.00	1	new_func1 [3]

```
$ gprof -b test_gprof gmon.out > analysis.txt
```

index	% time	self	children	called	name
<spontaneous>					
[1]	100.0	0.03	22.99		main [1]
		7.80	7.62	1/1	func1 [2]
		7.57	0.00	1/1	func2 [4]

		7.80	7.62	1/1	main [1]
[2]	67.0	7.80	7.62	1	func1 [2]
		7.62	0.00	1/1	new_func1 [3]

		7.62	0.00	1/1	func1 [2]
[3]	33.1	7.62	0.00	1	new_func1 [3]

		7.57	0.00	1/1	main [1]
[4]	32.9	7.57	0.00	1	func2 [4]

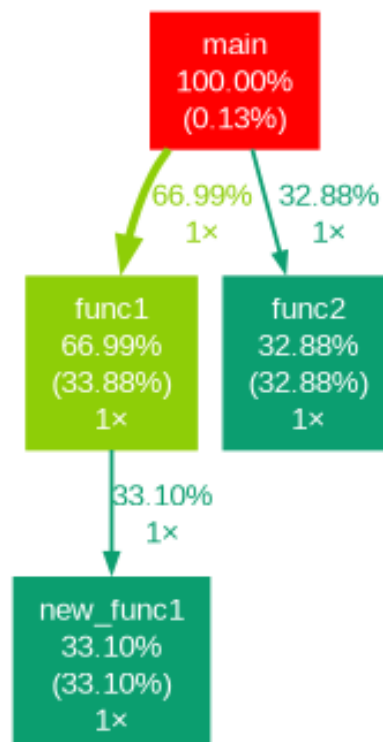
`gprof -pfunc1 -b test_gprof gmon.out > analysis.txt`

Flat profile:

Each sample counts as 0.01 seconds.

% time	cumulative seconds	self seconds	calls	self s/call	total s/call	name
33.88	7.80	7.80	1	7.80	15.42	func1
33.10	15.42	7.62	1	7.62	7.62	new_func1
32.88	22.99	7.57	1	7.57	7.57	func2
0.13	23.02	0.03				main

`gprof test_gprof gmon.out | python3 -m gprof2dot | dot -Tpng -o grafico.png`



```
gprof -a test_gprof gmon.out > analysis.txt
```

index	% time	self	children	called	name
[1]	100.0	0.01	0.08		<spontaneous>
		0.03	0.05	2/2	main [1]
					func1 [2]
[2]	88.9	0.03	0.05	2/2	main [1]
		0.03	0.05	2	func1 [2]
		0.05	0.00	1/1	new_func1 [3]
[3]	55.6	0.05	0.00	1/1	func1 [2]
		0.05	0.00	1	new_func1 [3]

```
$ gprof -b test_gprof gmon.out > analysis.txt
```

index	% time	self	children	called	name
[1]	100.0	0.01	0.08		<spontaneous>
		0.02	0.05	1/1	main [1]
		0.01	0.00	1/1	func1 [2]
					func2 [4]
[2]	77.8	0.02	0.05	1/1	main [1]
		0.02	0.05	1	func1 [2]
		0.05	0.00	1/1	new_func1 [3]
[3]	55.6	0.05	0.00	1/1	func1 [2]
		0.05	0.00	1	new_func1 [3]
[4]	11.1	0.01	0.00	1/1	main [1]
		0.01	0.00	1	func2 [4]

```
gprof -p -b test_gprof gmon.out > analysis_p.txt
```

Flat profile:

Each sample counts as 0.01 seconds.

% time	cumulative seconds	self seconds	calls	self ms/call	total ms/call	name
55.56	0.05	0.05	1	50.00	50.00	new_func1
22.22	0.07	0.02	1	20.00	70.00	func1
11.11	0.08	0.01	1	10.00	10.00	func2
11.11	0.09	0.01				main

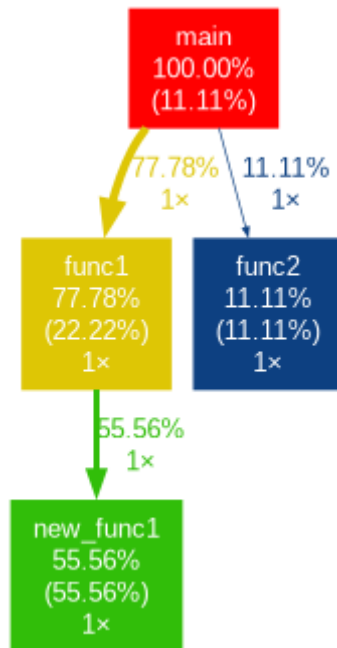
```
gprof -pfunc1 -b test_gprof gmon.out > analysis.txt
```

Flat profile:

Each sample counts as 0.01 seconds.

% time	cumulative seconds	self seconds	calls	self ms/call	total ms/call	name
100.00	0.02	0.02	1	20.00	20.00	func1

`gprof test_gprof gmon.out | python3 -m gprof2dot | dot -Tpng -o grafico.png`



Conclusiones sobre el uso del tiempo de las funciones

- La función que más tiempo propio (self) consume en ambas corridas es `new_func1`, seguida de cerca por `func1` y `func2` — algo esperable porque las tres funciones son, en esencia, el mismo tipo de bucle vacío ("for" sin trabajo real), y sus tiempos relativos dependen pura y exclusivamente de cuántas iteraciones tiene cada bucle, no de ninguna diferencia algorítmica.
- `main` casi no aparece con tiempo propio: su rol es solo llamar a `func1` y `func2`, así que su tiempo "children" es alto pero su self es mínimo.
- Detección de una anomalía en la corrida de Pía: los tiempos absolutos (0,01-0,09 s en total) resultaron muy bajos para 4 mil millones de iteraciones, muy por debajo de lo esperado según el apunte (~46 s) y muy por debajo de la corrida de Valentino. La causa más probable es que `gprof` corre dentro de una máquina virtual (VirtualBox).

¿Cómo se podría disminuir el tiempo de ejecución de este código?

- El "trabajo" de `func1`, `func2` y `new_func1` es un bucle vacío que no hace nada útil: la forma más directa de bajar el tiempo es simplemente reducir la cantidad de iteraciones al mínimo necesario para el propósito del ejercicio (perfilar), en vez de usar límites como `0xffffffff`.
- `func1` y `func2` son llamadas una después de la otra desde `main` y no dependen entre sí (no comparten datos), por lo que podrían ejecutarse en paralelo en dos hilos distintos.

5. Medición de red y videollamadas

Con herramienta iperf se realizo la medicion de ancho de banda entre dos maquinas en una misma red lan

```
C:\Users\Valentino\Downloads\iperf3.1.4_64>iperf3.exe -c 192.168.0.139
Connecting to host 192.168.0.139, port 5201
[ 4] local 192.168.0.242 port 58083 connected to 192.168.0.139 port 5201
[ ID] Interval           Transfer     Bandwidth
[ 4]  0.00-1.00      sec   23.6 MBytes  198 Mbits/sec
[ 4]  1.00-2.00      sec   27.4 MBytes  230 Mbits/sec
[ 4]  2.00-3.00      sec   30.5 MBytes  256 Mbits/sec
[ 4]  3.00-4.01      sec   30.5 MBytes  255 Mbits/sec
[ 4]  4.01-5.00      sec   27.2 MBytes  230 Mbits/sec
[ 4]  5.00-6.00      sec   30.6 MBytes  256 Mbits/sec
[ 4]  6.00-7.00      sec   32.2 MBytes  271 Mbits/sec
[ 4]  7.00-8.01      sec   34.8 MBytes  290 Mbits/sec
[ 4]  8.01-9.00      sec   32.1 MBytes  271 Mbits/sec
[ 4]  9.00-10.00     sec   34.6 MBytes  290 Mbits/sec
-----
[ ID] Interval           Transfer     Bandwidth
[ 4]  0.00-10.00     sec   304 MBytes  255 Mbits/sec
[ 4]  0.00-10.00     sec   304 MBytes  254 Mbits/sec
iperf Done.
```

```
tino@tino-Smart-Pro-Q7-i7:~/tp1-rendimiento$ iperf3 -s
Server listening on 5201 (test #1)
Accepted connection from 192.168.0.242, port 58082
[ 5] local 192.168.0.139 port 5201 connected to 192.168.0.242 port 58083
[ ID] Interval           Transfer     Bitrate
[ 5]  0.00-1.00      sec   22.8 MBytes  191 Mbits/sec
[ 5]  1.00-2.00      sec   27.8 MBytes  233 Mbits/sec
[ 5]  2.00-3.00      sec   30.2 MBytes  254 Mbits/sec
[ 5]  3.00-4.00      sec   30.5 MBytes  256 Mbits/sec
[ 5]  4.00-5.00      sec   27.2 MBytes  229 Mbits/sec
[ 5]  5.00-6.00      sec   30.6 MBytes  257 Mbits/sec
[ 5]  6.00-7.00      sec   32.2 MBytes  271 Mbits/sec
[ 5]  7.00-8.00      sec   34.4 MBytes  288 Mbits/sec
[ 5]  8.00-9.00      sec   32.4 MBytes  271 Mbits/sec
[ 5]  9.00-10.00     sec   34.6 MBytes  291 Mbits/sec
[ 5] 10.00-10.03     sec    640 KBytes  181 Mbits/sec
-----
[ ID] Interval           Transfer     Bitrate
[ 5]  0.00-10.03     sec   304 MBytes  254 Mbits/sec
receiver
```

medicion con fast.com

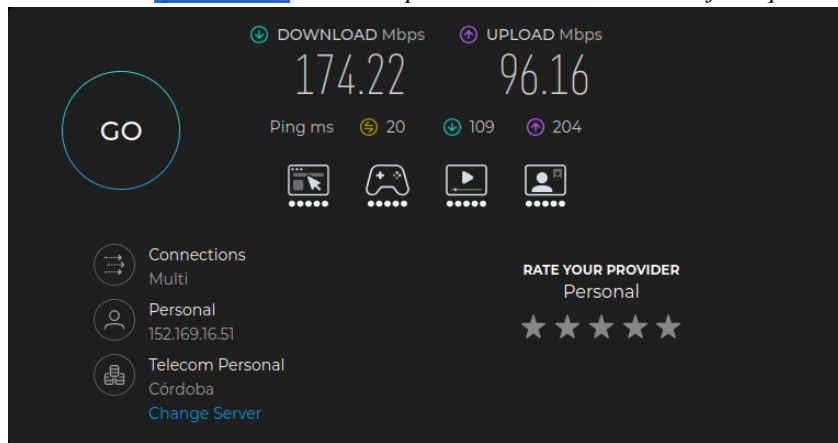


Tu velocidad de internet es de

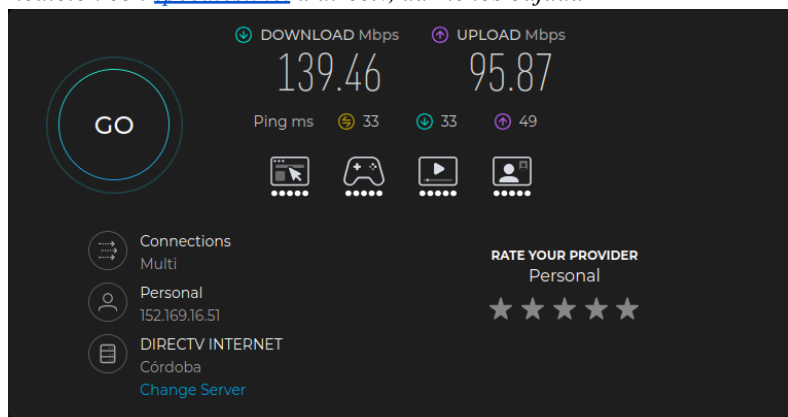
220 Mbps



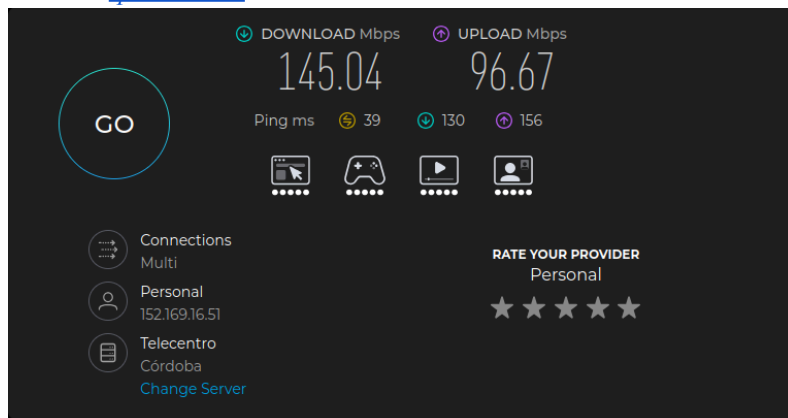
medicion con speedtest.net a telecom personal cordoba, menos bajada que servidores de netflix



medicion con speedtest.net a directv, da menos bajada



medicion speedtest.net contra telecentro



fast.com netflix



Tu velocidad de internet es de

190 Mbps



Latencia

Descargada

19 ms

Cargada

59 ms

Carga

Velocidad

78 Mbps

Cliente Cordoba, AR 152.169.16.51 Telecom - Arnet
Servidor(es) Córdoba, AR | Córdoba, AR | Munro, AR

Servidor	Bajada (Mbps)	Subida (Mbps)	Ping (ms)
(Ookla) Fibertel	174	96	20
(Ookla) DirecTV	139	95	33
(Ookla) Telecentro	145	96	39
(fast.com) Telecom - Arnet	190	78	19

Se midió la velocidad contra cuatro servidores de distintos proveedores. Los resultados variaron según el destino: la bajada osciló entre 139 y 190 Mbps y la latencia entre 19 y 39 ms. El mejor rendimiento se obtuvo contra servidores del propio proveedor (Telecom/Fibertel, ~190 Mbps y 19 ms), ya que el tráfico permanece dentro de la red del ISP sin saltar a redes externas. La subida se mantuvo estable en las mediciones de Ookla, confirmando que el límite lo impone el enlace de subida propio; fast.com reportó una subida menor por estar optimizado para medir bajada. Esto evidencia que el rendimiento de red es relativo al destino y a la ruta, no un valor absoluto.

Latencia por WiFi, red celular y cable

medición ping red wifi

```
tino@tino-Smart-Pro-Q7-i7:~/tp1-rendimiento$ ping -c 20 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=22.2 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=28.3 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=23.6 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=117 time=29.9 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=117 time=28.4 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=117 time=29.1 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=117 time=28.1 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=117 time=26.1 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=117 time=28.4 ms
64 bytes from 8.8.8.8: icmp_seq=10 ttl=117 time=26.4 ms
64 bytes from 8.8.8.8: icmp_seq=11 ttl=117 time=26.1 ms
64 bytes from 8.8.8.8: icmp_seq=12 ttl=117 time=27.8 ms
^C
--- 8.8.8.8 ping statistics ---
12 packets transmitted, 12 received, 0% packet loss, time 11014ms
rtt min/avg/max/mdev = 22.170/27.039/29.927/2.187 ms
```

medición ping red celu

```
tino@tino-Smart-Pro-Q7-i7:~/tp1-rendimiento$ ping -c 20 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=157 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=118 time=117 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=118 time=76.3 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=118 time=43.7 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=118 time=57.6 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=118 time=36.4 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=118 time=72.6 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=118 time=192 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=118 time=45.7 ms
64 bytes from 8.8.8.8: icmp_seq=10 ttl=118 time=44.5 ms
64 bytes from 8.8.8.8: icmp_seq=11 ttl=118 time=69.6 ms
64 bytes from 8.8.8.8: icmp_seq=12 ttl=118 time=43.0 ms
64 bytes from 8.8.8.8: icmp_seq=13 ttl=118 time=41.9 ms
^C
--- 8.8.8.8 ping statistics ---
13 packets transmitted, 13 received, 0% packet loss, time 12009ms
rtt min/avg/max/mdev = 36.360/76.746/191.800/47.146 ms
```

medicion ping red ethernet en windows porque con linux no andaba

```
C:\Windows\system32>ping -n 20 8.8.8.8

Haciendo ping a 8.8.8.8 con 32 bytes de datos:
Respuesta desde 8.8.8.8: bytes=32 tiempo=16ms TTL=117
Respuesta desde 8.8.8.8: bytes=32 tiempo=17ms TTL=117
Respuesta desde 8.8.8.8: bytes=32 tiempo=21ms TTL=117
Respuesta desde 8.8.8.8: bytes=32 tiempo=21ms TTL=117
Respuesta desde 8.8.8.8: bytes=32 tiempo=23ms TTL=117
Respuesta desde 8.8.8.8: bytes=32 tiempo=19ms TTL=117
Respuesta desde 8.8.8.8: bytes=32 tiempo=22ms TTL=117
Respuesta desde 8.8.8.8: bytes=32 tiempo=21ms TTL=117
Respuesta desde 8.8.8.8: bytes=32 tiempo=20ms TTL=117
Respuesta desde 8.8.8.8: bytes=32 tiempo=20ms TTL=117
Respuesta desde 8.8.8.8: bytes=32 tiempo=20ms TTL=117
Respuesta desde 8.8.8.8: bytes=32 tiempo=21ms TTL=117
Respuesta desde 8.8.8.8: bytes=32 tiempo=21ms TTL=117

Estadísticas de ping para 8.8.8.8:
    Paquetes: enviados = 13, recibidos = 13, perdidos = 0
    (0% perdidos),
    Tiempos aproximados de ida y vuelta en milisegundos:
        Mínimo = 16ms, Máximo = 23ms, Media = 20ms
```

Se midió la latencia en tres tipos de conexión. Los resultados: cable 20 ms, WiFi 27 ms y datos móviles 77 ms de media. La diferencia entre cable y WiFi fue menor a la esperada, ya que la latencia hacia un servidor externo está dominada por la ruta de internet, no por el enlace local; la ventaja del cable se apreciaría midiendo contra el router local. El dato más significativo fue el jitter (variabilidad): ~2 ms en redes fijas frente a 47 ms en datos móviles. Esta inestabilidad de la red celular es la principal causa de cortes y pérdida de calidad en videollamadas, más que la latencia media en sí. La medición por cable se realizó en Windows por un problema de DHCP en la interfaz cableada de Linux.

Tabla completa (5.3)

Conexión	Media (ms)	Jitter (ms)	Dónde
Cable	20 (mín 16, máx 23)	~2	Windows
WiFi	27,04	2,19	Linux
Celular	76,75	47,15	Linux

Videollamada de Google Meet y perturbación de la red

meet

RETRASO EN LA CONEXIÓN

Los valores muy elevados o las grandes variaciones pueden reducir la calidad de la llamada



11:26:59

— Retraso en la conexión 29 ms

-- Valor recomendado < 300 ms

videos de youtube en 2 compus y actualización de videojuego

RETRASO EN LA CONEXIÓN

Los valores muy elevados o las grandes variaciones pueden reducir la calidad de la llamada



11:31:42

— Retraso en la conexión 40 ms

-- Valor recomendado < 300 ms

Se intentó perturbar una videollamada de Google Meet saturando la red con dos reproducciones de YouTube y la actualización de un videojuego. El retraso de conexión subió de 29 ms a 40 ms, pero se mantuvo muy por debajo del umbral recomendado (<300 ms), por lo que la calidad no se degradó de forma perceptible. Esto se explica porque el tráfico generado satura principalmente la descarga, mientras que una videollamada es sensible sobre todo a la subida (el flujo saliente de audio/video) y al jitter. Además, el ancho de banda disponible (190 Mbps) absorbe con holgura ese tráfico. Para degradar la llamada de forma significativa habría que saturar el enlace de subida.

perturbando con iperf en modo subida



Se perturbó una videollamada de Google Meet saturando la red en dos escenarios. Con tráfico de descarga (dos videos de YouTube + actualización de un videojuego), el retraso subió levemente de 29 ms a 40 ms, sin degradación perceptible, porque el ancho de banda de bajada (190 Mbps) absorbía ese tráfico. En cambio, al saturar la red local con iperf3 (20 conexiones paralelas), el retraso se disparó a 98 ms —más del triple del valor base—, evidenciando un pico abrupto coincidente con el inicio de la saturación. Esto demuestra que el cuello de botella era la red local (WiFi), y que la videollamada es sensible a la saturación del medio compartido. En todos los casos el retraso se mantuvo bajo el umbral de Meet (<300 ms), por lo que hubo degradación pero no caída de la llamada.